DATABASE DESIGN PROJECT

Online Store Relational Database

A fully structured relational database for an online store, built using **PostgreSQL**. This project demonstrates core and advanced SQL concepts through a complete, production-ready schema.

7	15+	837
Normalized Tables	SQL Concepts	Lines of SQL
Fully structured with constraints and foreign keys	From basic SELECT to advanced triggers and procedures	Complete, executable script

`Concepts covered: Database Design · Normalization · Data Types · SELECT/WHERE/ORDER BY · DML · Aggregate Functions · GROUP BY/HAVING · All JOIN types · UNION · Subqueries · Indexes · Views · Transactions · Stored Procedures · Functions · Triggers · Date & String Functions · EXPLAIN`

SCHEMA DESIGN

Database Schema → 7 Tables

customers

Stores customer personal details : name, email, phone, address, date of birth

categories

Product category information; organizes products into logical groups

products

Product listings with name, price, stock quantity, and category FK

orders

`Customer orders with full status tracking: pending → confirmed → shipped → delivered → cancelled`

order_items

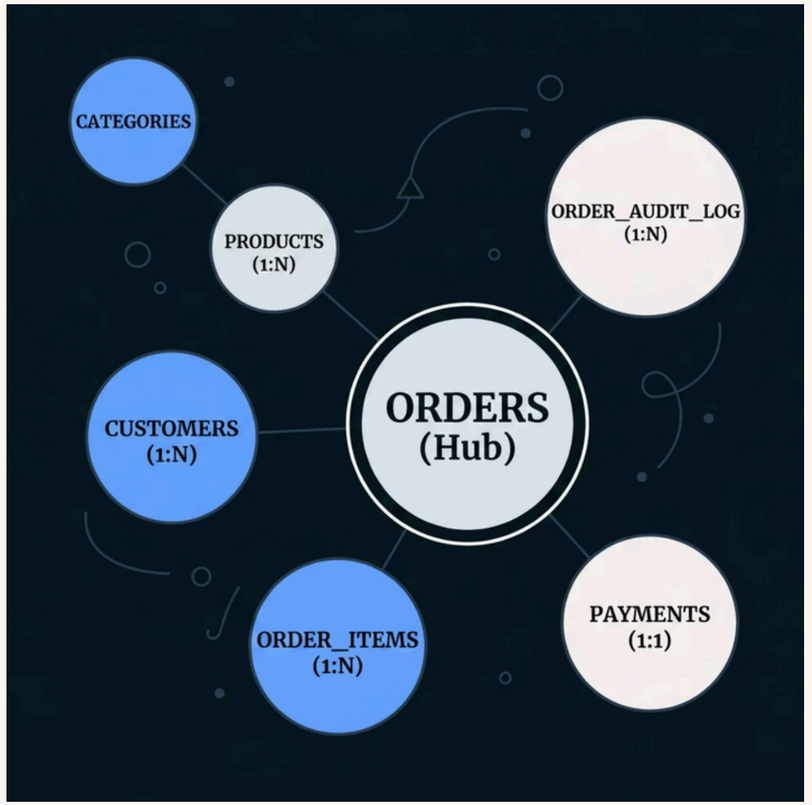
Junction table resolving the M:N relationship between orders and products

payments

Payment records with method, amount, and status for each order

order_audit_log

Auto-populated by trigger : logs every order status change with timestamps



RELATIONSHIPS

ER Diagram & Relationships

1

1:N

customers → orders
One customer places many orders

2

1:N

categories → products
One category contains many products

3

M:N

orders ↔ products
Resolved via order_items junction table

4

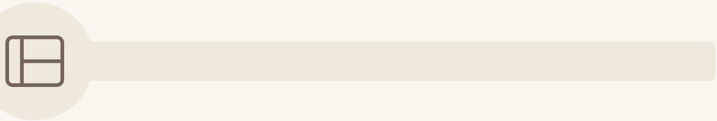
1:1

orders → payments
One order has exactly one payment record

The **order_items** table is the architectural centerpiece - it resolves the many-to-many relationship between orders and products, storing quantity and price per line item. The **order_audit_log** is the only table never written to manually; it is exclusively populated by a database trigger.

DATA INTEGRITY

Normalization - 1NF, 2NF & 3NF

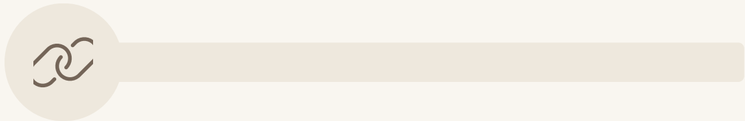


1NF First Normal Form

Rule: No repeating groups; every column holds exactly one atomic value.
Applied: order_items is a separate table instead of stuffing product1, product2, product3 columns inside the orders table. Each row represents one product in one order.

2NF Second Normal Form

Rule: No partial dependencies - non-key columns must depend on the *whole* primary key.
Applied: Customer details are **not** inside the orders table. Only customer_id (FK) is stored. Full customer info lives exclusively in the customers table.



3NF Third Normal Form

Rule: No transitive dependencies - columns depend only on the primary key, not on other non-key columns.
Applied: category_name is **not** repeated in every product row. It lives in the categories table. If a category name changes, only one row needs updating.

SCHEMA DETAIL

Data Types Used

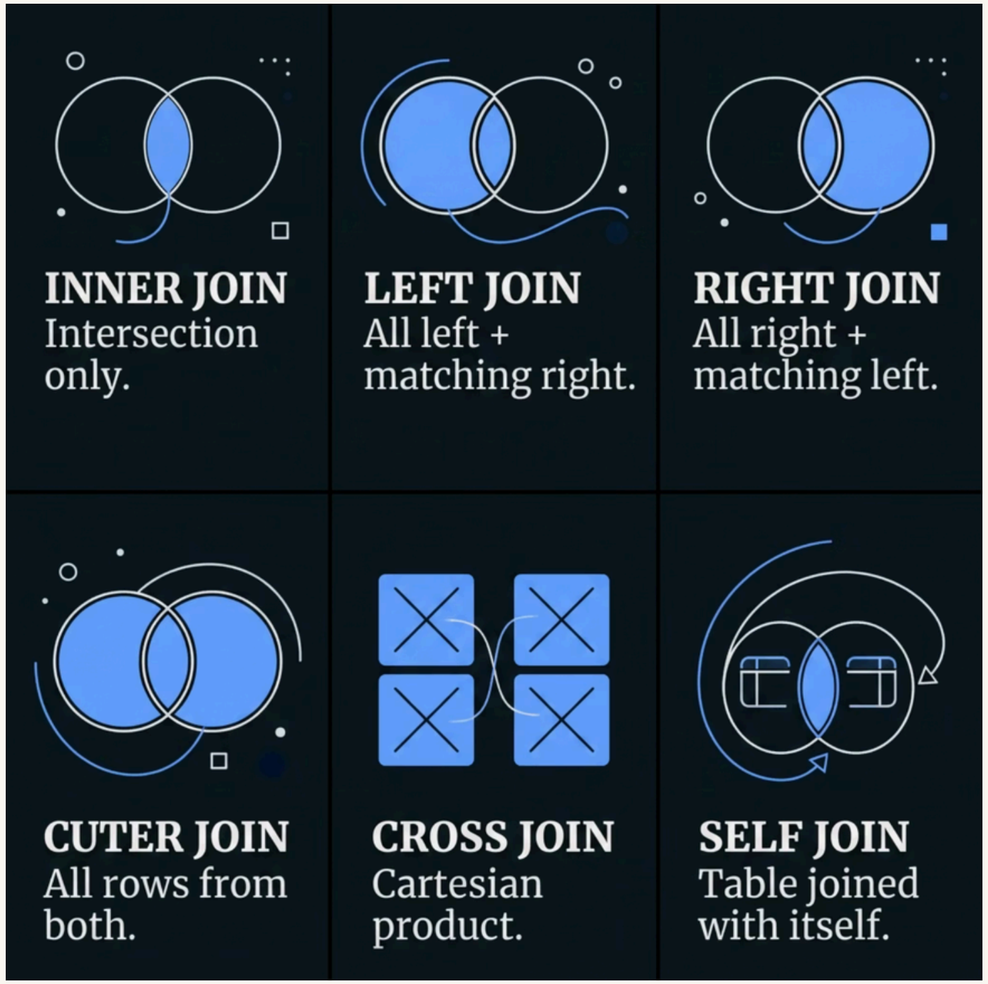
Every column's data type was chosen deliberately for correctness, storage efficiency, and global compatibility.

Data Type	Used For	Storage	Why This Type
`SERIAL`	Primary Keys	4 bytes	Auto-incrementing, no manual ID management needed
`VARCHAR(n)`	Names, Email, Phone	Variable	Variable length, saves space vs fixed CHAR
`TEXT`	Descriptions, Addresses	Variable	No length limit for long, unpredictable content
`DECIMAL(10,2)`	Price, Amount	Exact	Exact decimal precision, never use FLOAT for money
`INT`	Quantity, Stock	4 bytes	Whole numbers for countable values
`TINYINT`	Age	1 byte	Max 255, more than enough for age values
`DATE`	Date of Birth	4 bytes	Date only, no time component needed
`TIMESTAMPTZ`	created_at, order_date	8 bytes	Date + Time + Timezone for global accuracy

⚠ **Critical rule:** Always use DECIMAL(10,2) for monetary values. FLOAT introduces rounding errors that are unacceptable in financial data.

QUERY POWER

All 6 JOIN Types Implemented



INNER JOIN

Returns only rows matching in both tables. *Example:* orders joined with customers to show order details with customer names.

LEFT JOIN

All rows from left table, NULLs where no match on right. *Example:* all customers including those with no orders yet.

RIGHT JOIN

All rows from right table, NULLs where no match on left side.

FULL OUTER JOIN

All rows from both tables, NULLs wherever no match exists on either side.

CROSS JOIN

Every row from table A combined with every row from table B, all possible combinations (Cartesian product).

SELF JOIN

A table joined with itself. *Example:* find customers from the same city as another customer.

PROGRAMMABILITY

Stored Procedures & Functions

Key distinction: A **Stored Procedure** performs an action (side effects, no return value). A **Function** always returns a value and can be used inside a SELECT statement.

Stored Procedures — Perform Actions

1

place_order()

Checks stock → Creates order → Adds items → Reduces stock → Records payment. All in one atomic call:

```
CALL place_order(3, 5, 2, 'upi');
```

2

update_order_status()

Updates an order's status field. Raises a descriptive error if the specified order_id is not found in the database.

Functions — Return Values

1

get_customer_age()

Accepts a customer_id, calculates age from date_of_birth using PostgreSQL's AGE() function, and returns an integer.

2

get_category_revenue()

Joins order_items + products + orders, excludes cancelled orders, and returns total revenue per product category.

AUTOMATION

Triggers — Automatic Database Behavior

Trigger 1 Order Status Audit Log

Fires: AFTER UPDATE on the orders table
What it does: Every time an order status changes, automatically inserts a record into order_audit_log capturing the old status, new status, and a precise timestamp. No manual insert is ever needed, the trigger handles it entirely.
Demo:

```
UPDATE orders SET status = 'shipped'
WHERE order_id = 4;

SELECT * FROM order_audit_log;
```

Trigger 2 Prevent Deletion of Delivered Orders

Fires: BEFORE DELETE on the orders table
What it does: If someone attempts to delete a delivered order, the trigger raises an error and cancels the deletion entirely, protecting completed transaction records from accidental or malicious removal.
Demo:

```
DELETE FROM orders WHERE order_id = 1;
-- ERROR: Cannot delete a delivered order
```

Triggers enforce business rules at the **database level**. They fire regardless of which application, user, or script modifies the data, making them the strongest form of rule enforcement.

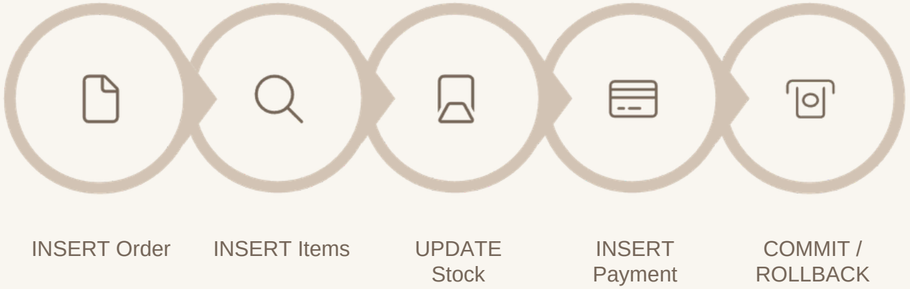
ADVANCED FEATURES

Views, Transactions & Indexes

Views → 3 Created

Transactions → ACID Compliance

All related operations in place_order() are wrapped in a BEGIN...COMMIT block:



If any step fails, a ROLLBACK is issued. No partial data is ever saved, guaranteeing data integrity.

Indexes & Query Optimization

idx_customers_email

customers(email) : fast customer lookup by email address

idx_products_name

products(product_name) : fast product search by name

idx_orders_status

orders(status) : fast filtering by order status

idx_order_items_composite

order_items(order_id, product_id) : speeds up JOIN operations on the junction table

EXPLAIN ANALYZE reveals whether a query uses an **Index Scan** (jumps directly to matching rows, fast) or a **Sequential Scan** (reads every row, slow on large tables).

vw_order_summary

Joins customers + orders + payments into one clean, queryable result set.

vw_product_inventory

Shows stock status using a CASE statement: **In Stock** / **Low Stock** / **Out of Stock**

vw_customer_summary

Total orders placed and total amount spent per customer, aggregated in one view.

LIVE DEMO & SUMMARY

Live Demo Plan & Project Summary

Live Demo 8 Steps

01

Show all 7 tables

Browse data in pgAdmin to confirm schema and seed data

02

INNER JOIN

Orders joined with customer names

03

LEFT JOIN

Customers with no orders showing as NULL

04

Aggregate Query

Revenue per category using GROUP BY

05

CALL place_order()

Stored procedure → full transaction in one call

06

Trigger Demo

Update order status → audit log auto-populated

07

Views

```
SELECT * FROM vw_product_inventory
```

08

EXPLAIN ANALYZE

Index scan vs sequential scan comparison

Project Summary

7 Normalized Tables

Proper constraints, foreign keys, and referential integrity throughout the entire schema

All JOIN Types

Demonstrated with real Kerala-based customer data across all 6 join variants

Full Programmability

Stored procedures, functions, triggers, and views all implemented and tested

Data Integrity

Transactions with COMMIT and ROLLBACK; trigger-enforced business rules

Built entirely in **PostgreSQL** following industry database design best practices, from normalization through query optimization with EXPLAIN ANALYZE.
🔗 GitHub: https://github.com/thearunraju/online-store-db